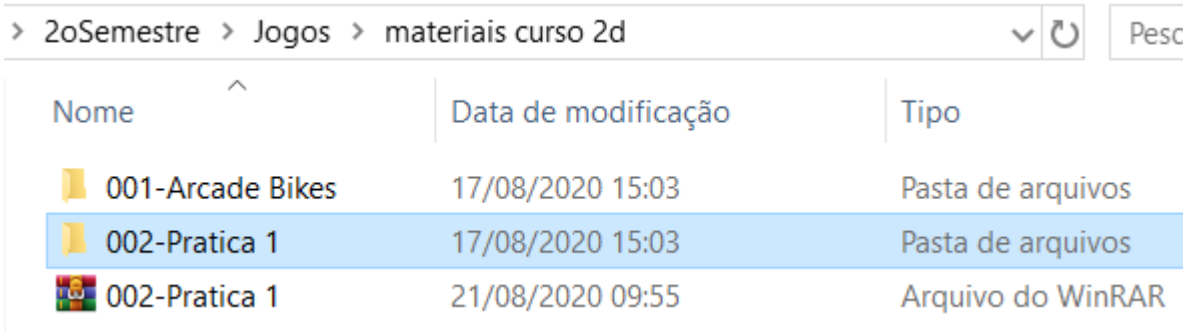
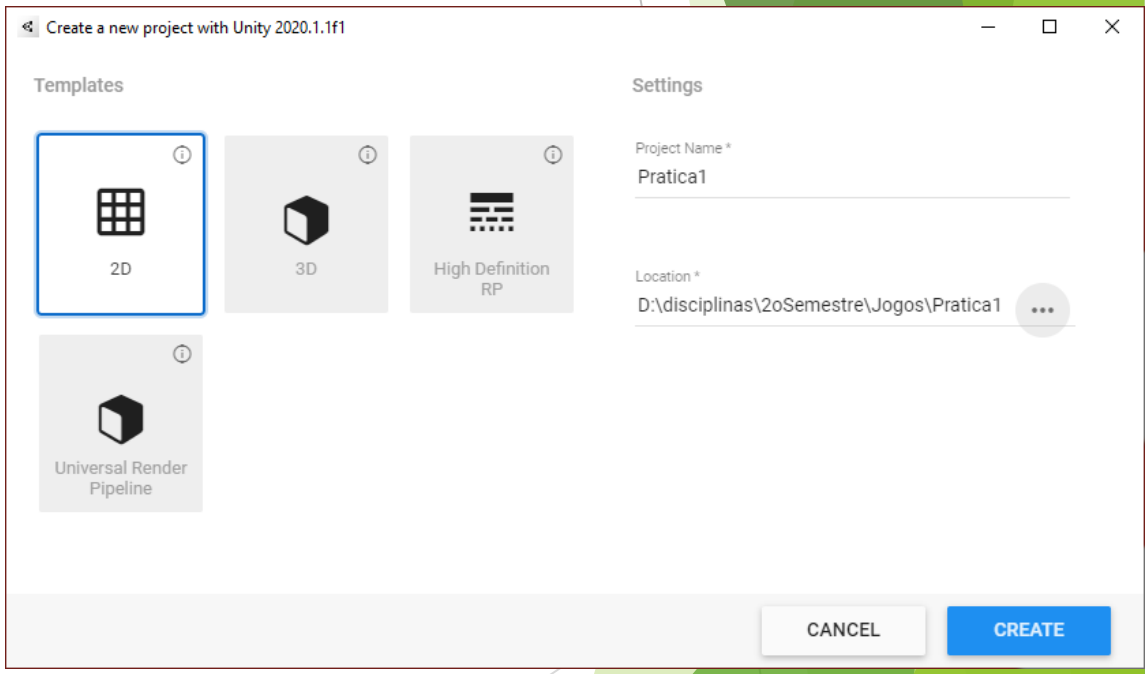
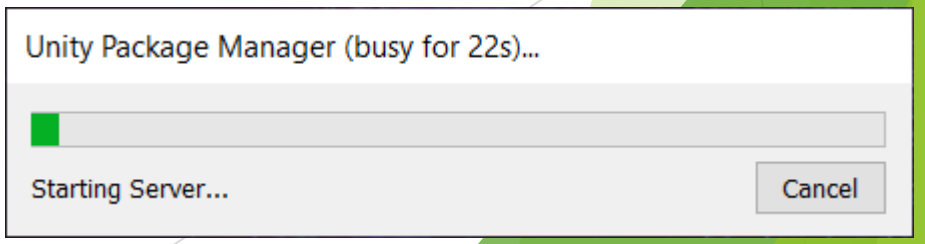
Prática 1: um primeiro projeto

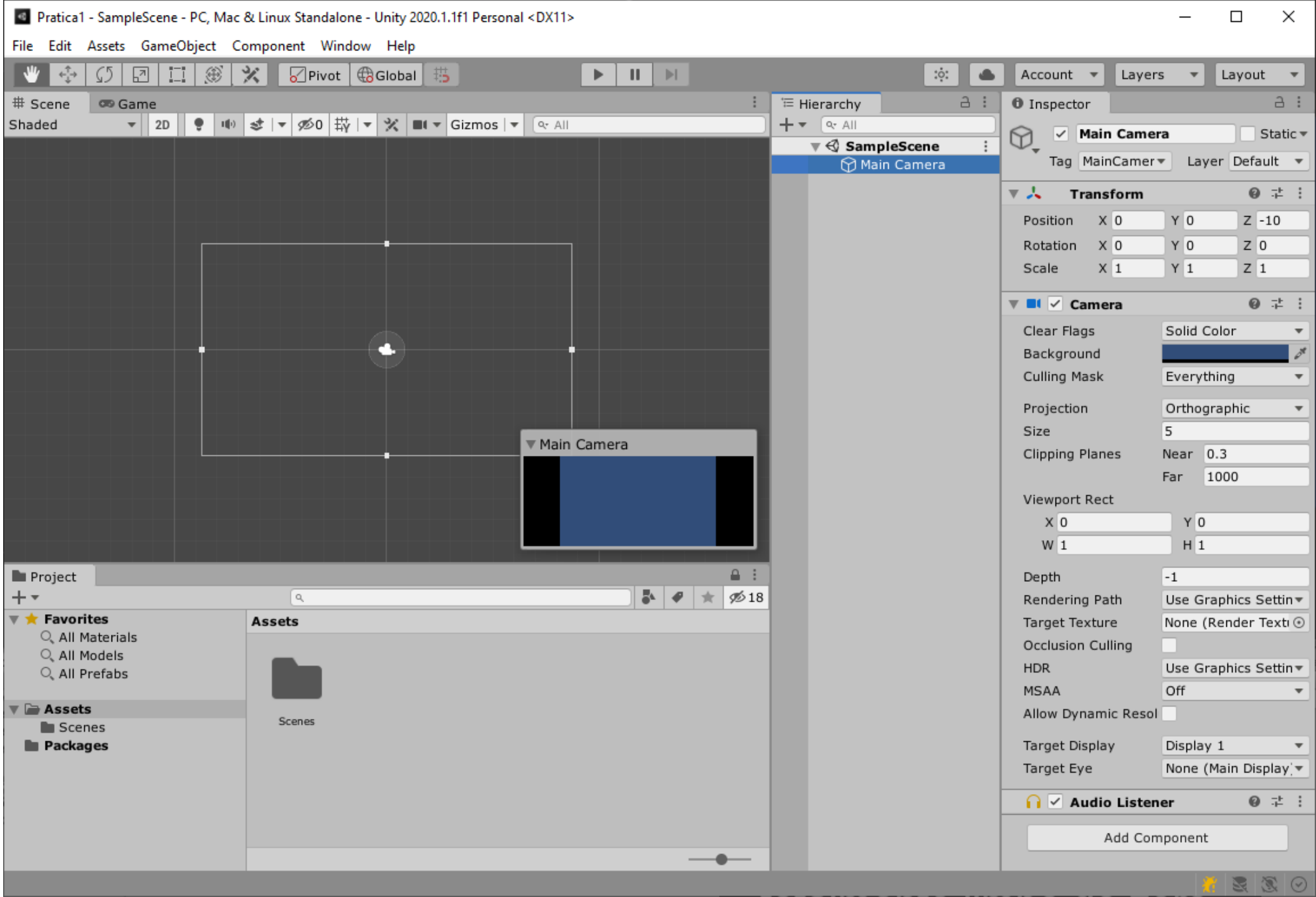
- Copie no seu disco a pasta de assets fornecida pelo professor para esta atividade (002-Pratica 1.rar) e o descompacte em uma pasta, por exemplo, c:\temp ou outra pasta onde esteja guardando os materiais da disciplina.
- No Unity Hub, selecione o item Projects e clique no botão [NEW]. Se estiver no Unity, você pode usar a opção de menu File | New Project.
- Em ambos os casos aparecerá a tela ao lado.
- Selecione o quadro 2D, digite o nome do seu projeto (por exemplo Pratica1) e selecione uma localização para ele; poderá, por exemplo, criar uma pasta Pratica1 na pasta onde está armazenado os materiais desta disciplina. Pressione o botão [CREATE] e tenha paciência, pois Unity carregará vários arquivos, importará assets padrão e organizará as pastas do projeto.
- Teremos, então, um novo projeto no ambiente de desenvolvimento do Unity, que pode ser visualizado no próximo slide.





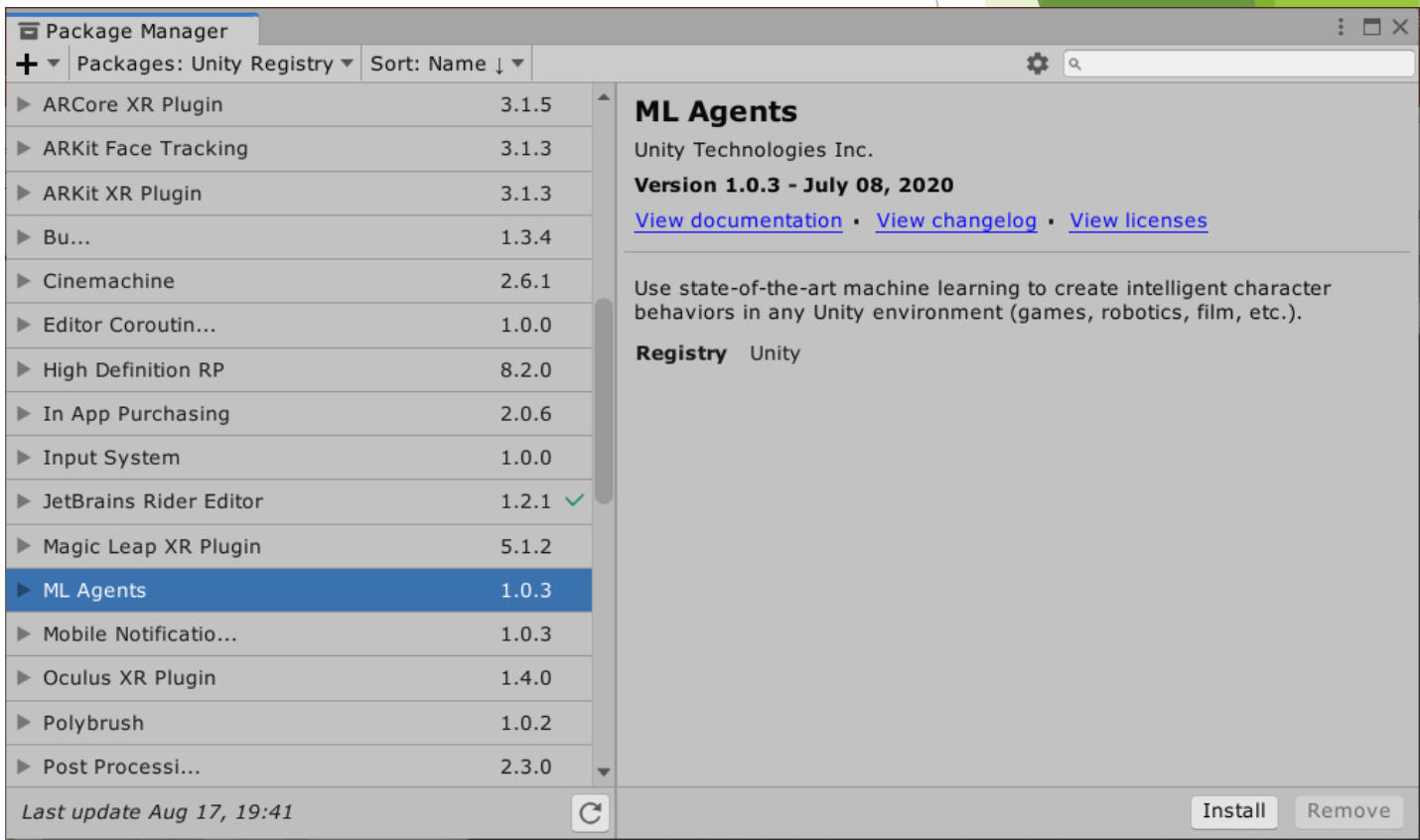


Prática 1


Pratica1 - SampleScene - PC, Mac & Linux Standalone - Unity 2020.1.1f1 Personal <DX11>
File Edit Assets GameObject Component Window Help
Pivot Global
Account Layers Layout
Scene Game
Shaded 2D Gizmos All
Main Camera
Hierarchy
SampleScene
Main Camera
Inspector
Main Camera Static
Tag MainCamer Layer Default
Transform
Position X 0 Y 0 Z -10
Rotation X 0 Y 0 Z 0
Scale X 1 Y 1 Z 1
Camera
Clear Flags Solid Color
Background
Culling Mask Everything
Projection Orthographic
Size 5
Clipping Planes Near 0.3
Far 1000
Viewport Rect
X 0 Y 0
W 1 H 1
Depth -1
Rendering Path Use Graphics Settin
Target Texture None (Render Texti
Occlusion Culling
HDR Use Graphics Settin
MSAA Off
Allow Dynamic Resol
Target Display Display 1
Target Eye None (Main Display
Audio Listener
Add Component
Project
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Packages
Assets
Scenes

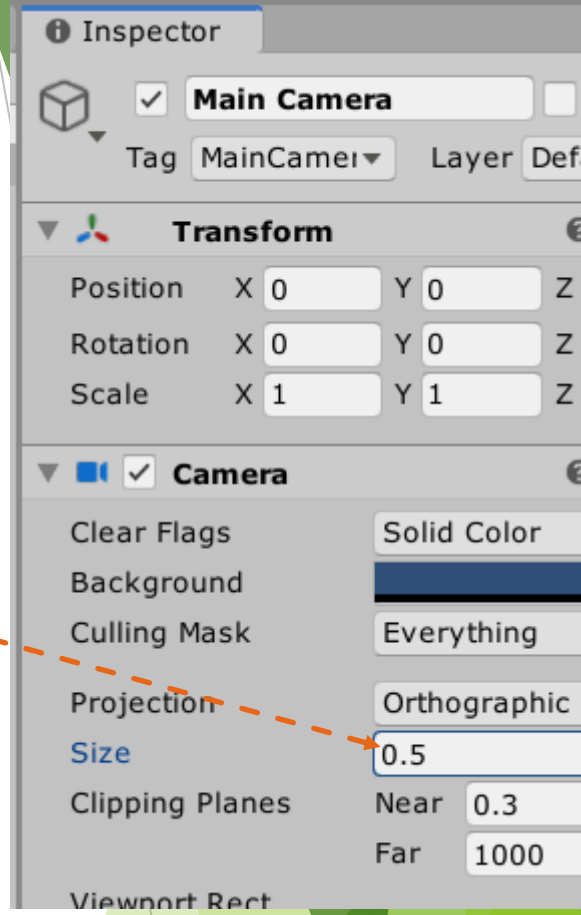
Prática 1

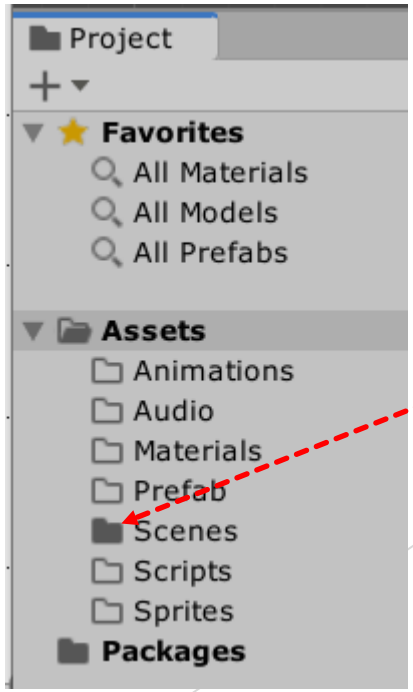
- Quando Unity carrega um projeto pela primeira vez, ele inclui uma série de pacotes que não serão necessários e adicionam tempo de compilação, à toa.
- Acessando o menu Windows | Package Manager, podemos ver os pacotes instalados e aqueles disponíveis para instalação, pois continuamente se criam novos pacotes para permitir o uso do Unity com novas tecnologias, como Machine Learning, Realidade Mista, novos dispositivos de entrada, dentre outros.
  (Vídeo sobre Unity e VR → https://www.youtube.com/watch?v=Hnoad3DM_pA)
- Se você abrir a lista de tipos de pacotes, ao lado do símbolo + na parte superior dessa janela, verá que tem uma opção In Project.
- Selecione essa opção e apenas os pacotes adicionados ao seu projeto serão exibidos.
- O pacote Unity Collaborate permite o desenvolvimento por grupos online.
- Você poderá remover alguns pacotes como, por exemplo, JetBrains Rider Editor, caso não use esse editor para desenvolver os scripts do seu jogo.
- Ao terminar, feche essa janela.



Prática 1

- Na cena ou hierarquia, selecione a câmera principal e mude a sua propriedade Camera | Size de 5 para 0.5. Assim, veremos a cena mais próxima e os objetos nela colocados parecerão maiores.
- Em seguida, vamos organizar nosso projeto, criando pastas adicionais. Criá-las agora facilitará a estruturação e arquitetura do projeto e evitará eventuais problemas.
- Na janela Project, clique com o botão direito, selecione a opção Create | Folders e crie as pastas abaixo:
  - Animations
  - Materials
  - Audio
  - Prefab
  - Scripts
  - Sprites



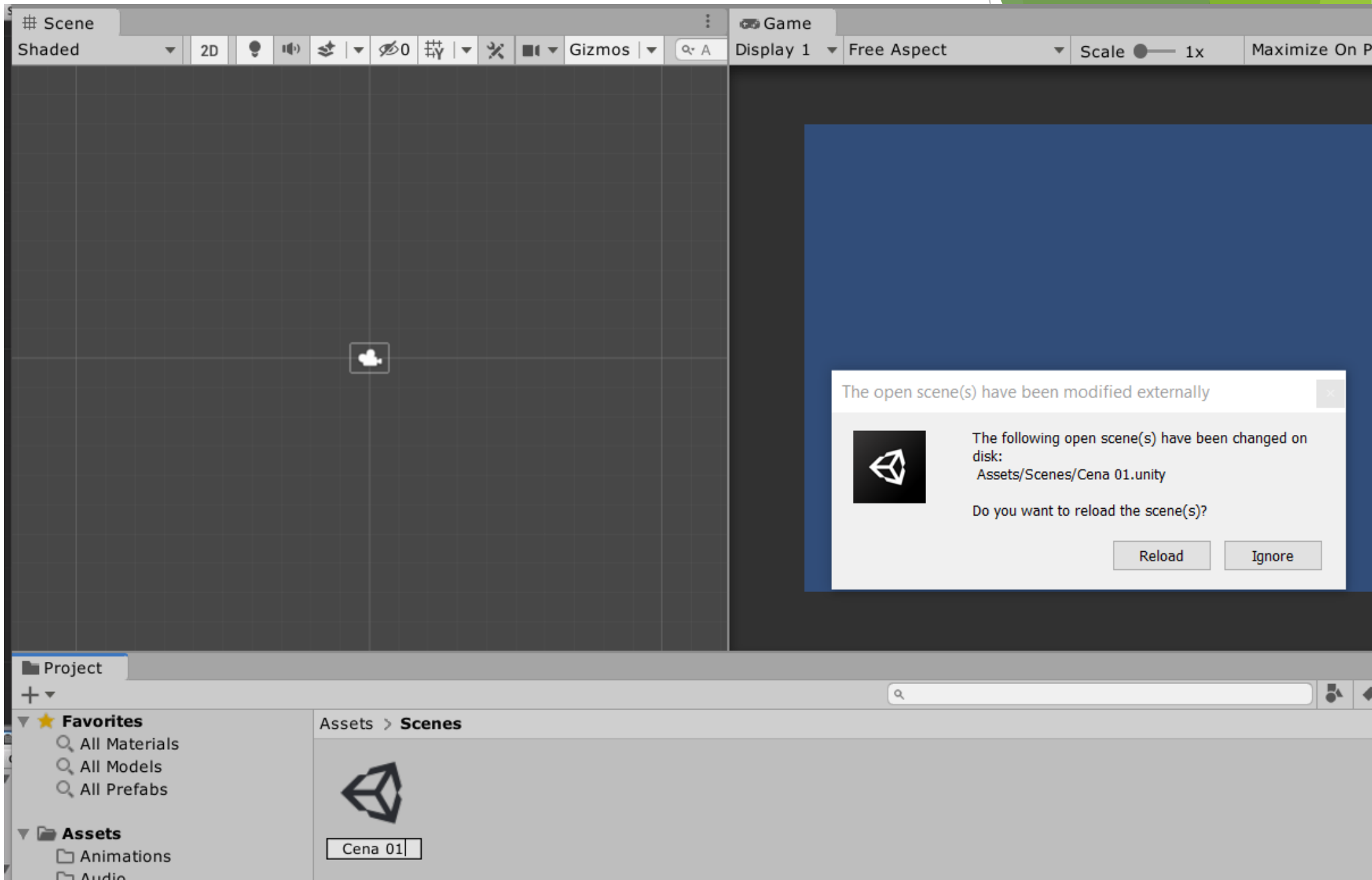


O ícone preenchido indica que há arquivos nesta pasta.

- Se você criar uma pasta no local errado (por exemplo, dentro de outra), basta arrastá-la para fora e posicionar o cursor em Assets até aparecer uma linha azul sobre a pasta Assets e largar a pasta que movimenta nesse local.

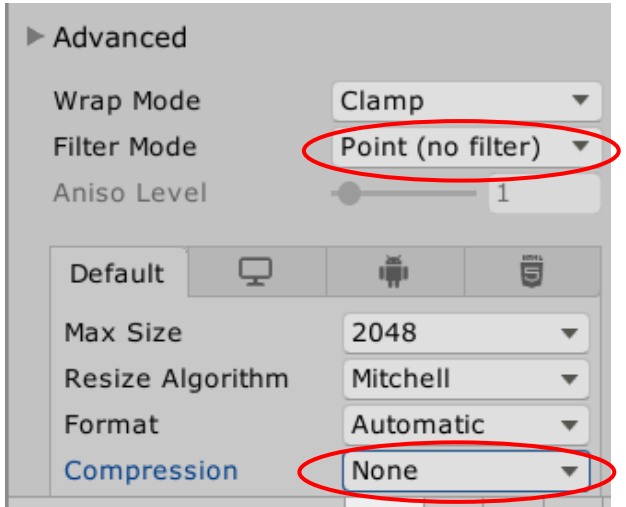
Prática 1

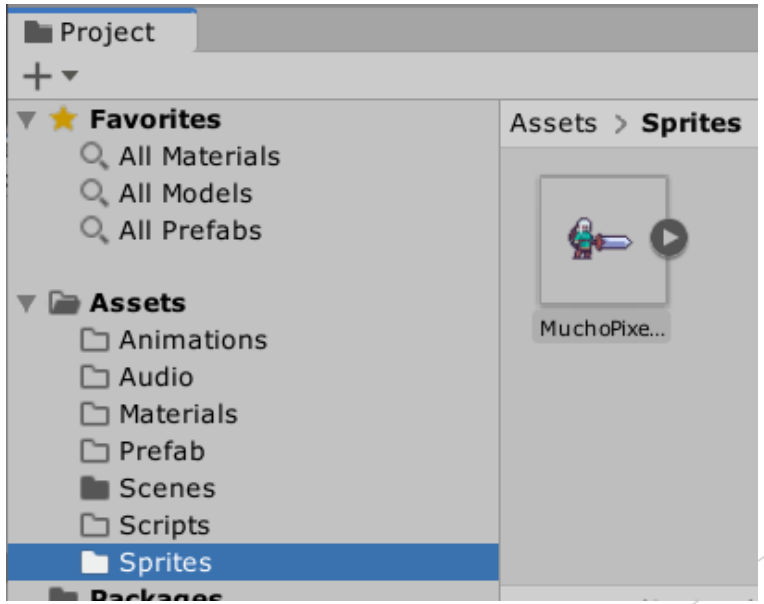
- Clique no nome da cena e a mude para Cena 01.
- Unity perguntará se deve salvar e recarregar, pressione [Reload] para fazer isso.
- Na parte superior da janela do Unity teremos o nome do projeto e da cena aberta, bem como da plataforma de build que estamos usando no momento.

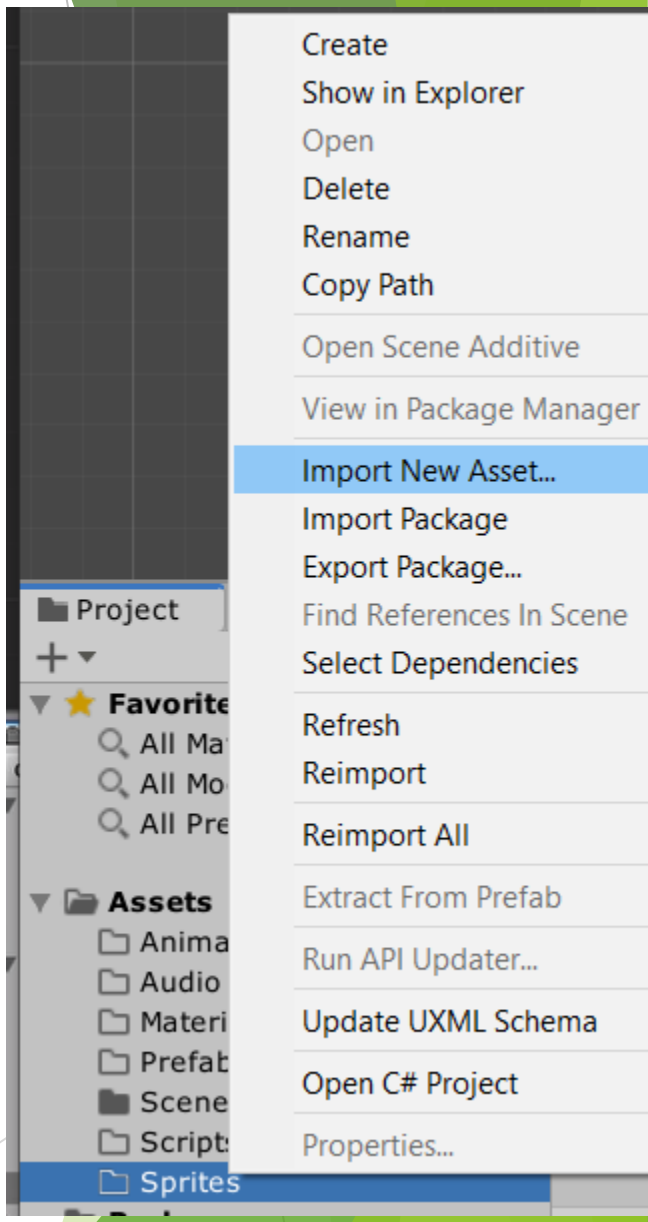


Prática 1

- Vamos agora importar assets para nosso jogo.
- Clique na pasta Sprites da janela Project com o botão direito e, sem seguida, selecione a opção [Import New Asset...] no menu suspenso que aparecerá.
- Aparecerá uma janela de seleção de arquivo. Direcione para a pasta onde salvou o material deste jogo e selecione o arquivo MuchosPixels_Player.png.
- Clique no botão [Import] para trazê-lo ao seu projeto. O arquivo será copiado fisicamente na pasta Assets|Sprites do seu projeto e poderá ser tratado no jogo.
- Clique nesse asset para selecioná-lo no Inspector.
- Modifique a propriedade Filter Mode para Point (no filter) e a propriedade Compression para None:
- Assim, teremos uma imagem perfeita.

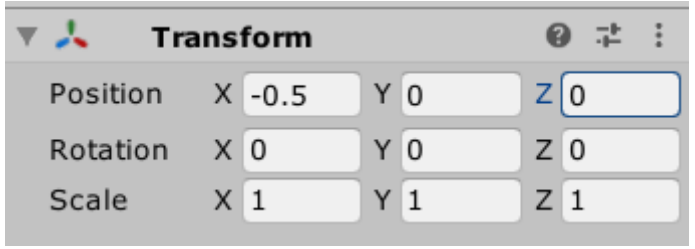


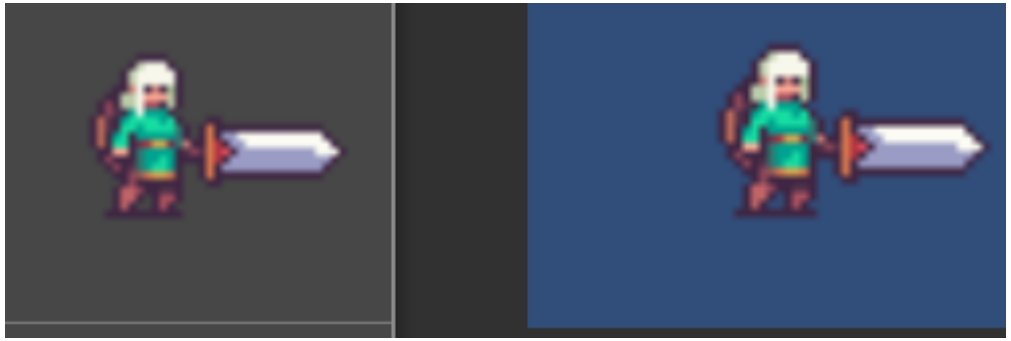


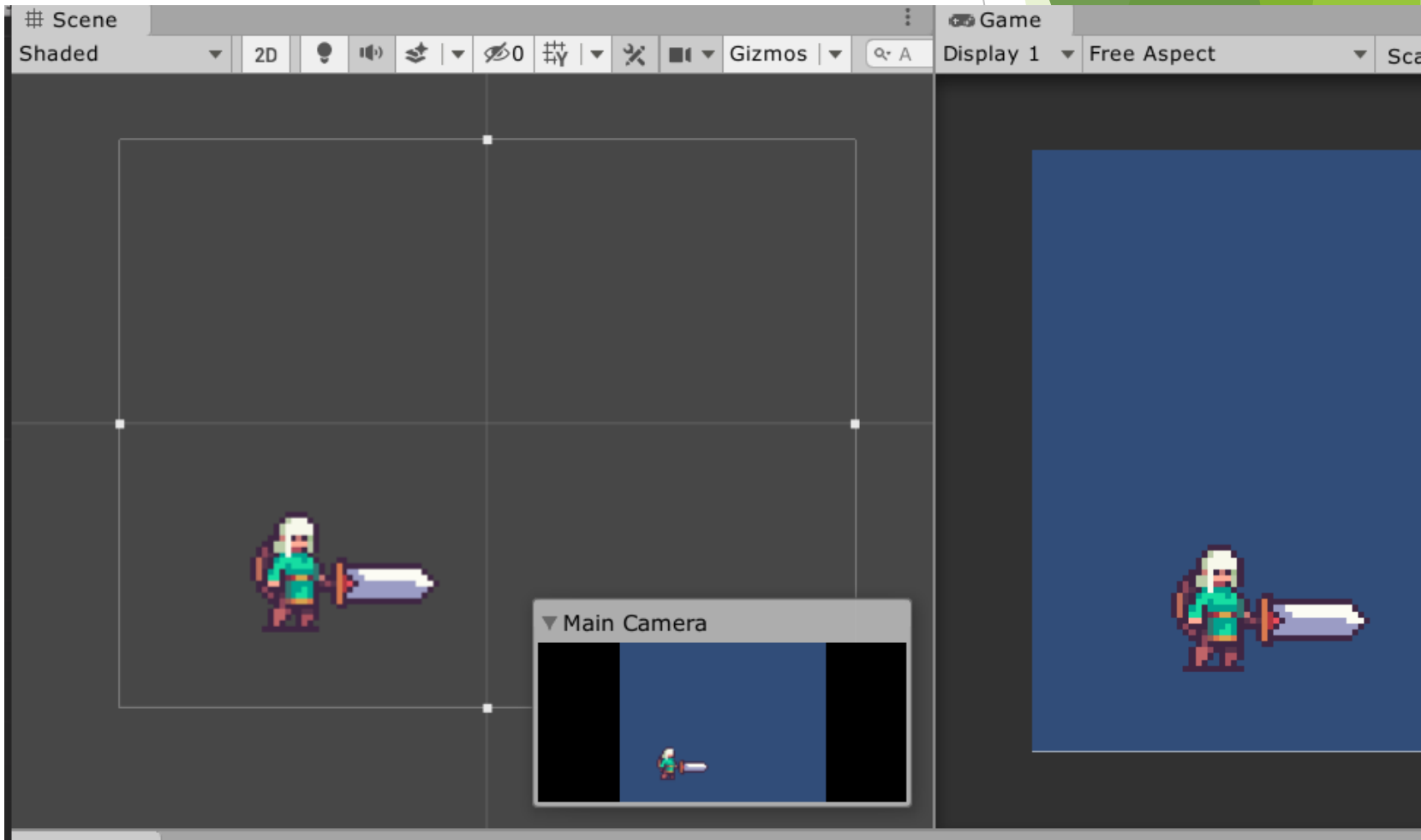


Prática 1

- Arraste esse objeto para a cena.
- Se você modificar novamente o filtro ou a compressão, verá que a imagem poderá ficar borrada.
- Portanto, para imagens pixeladas, é melhor trabalhar sem filtro e sem compressão. Deixe o personagem sem filtro e sem compressão.
- Aplique zoom na janela Scene para ver melhor o personagem e, ao mesmo tempo, veja-o na janela Game, como ao lado.
- Clique no personagem na janela Scene e mude, no seu Inspector, Position.X para -0.5 e Position.Y para 0.

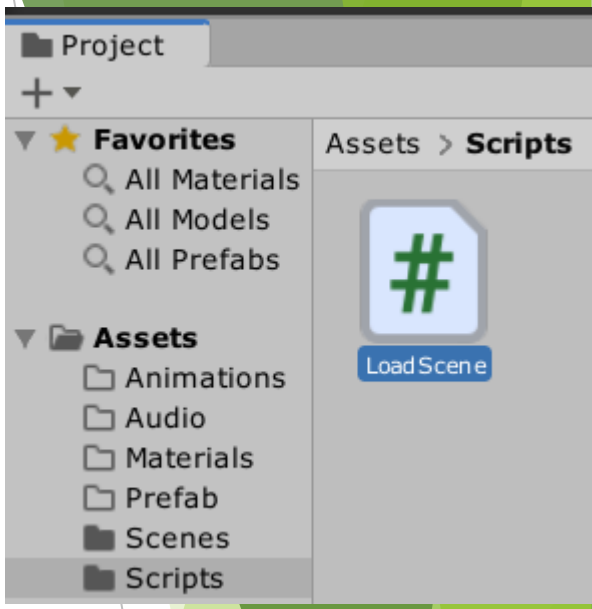


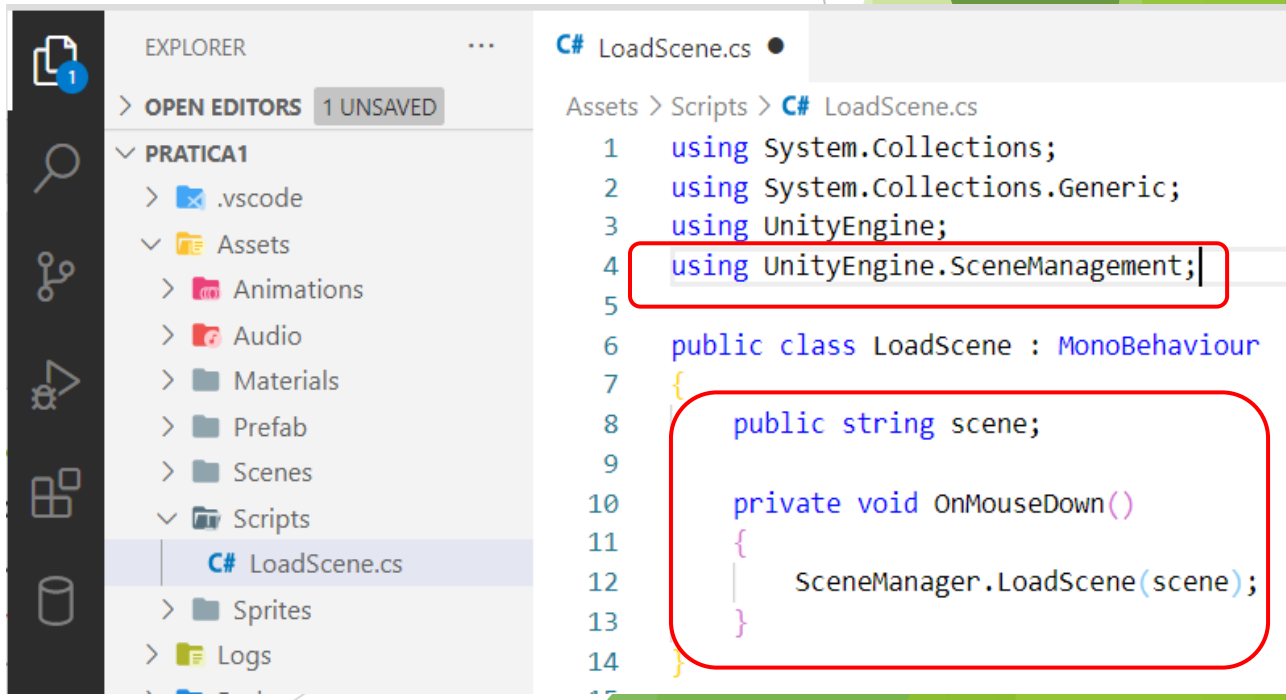




Prática 1

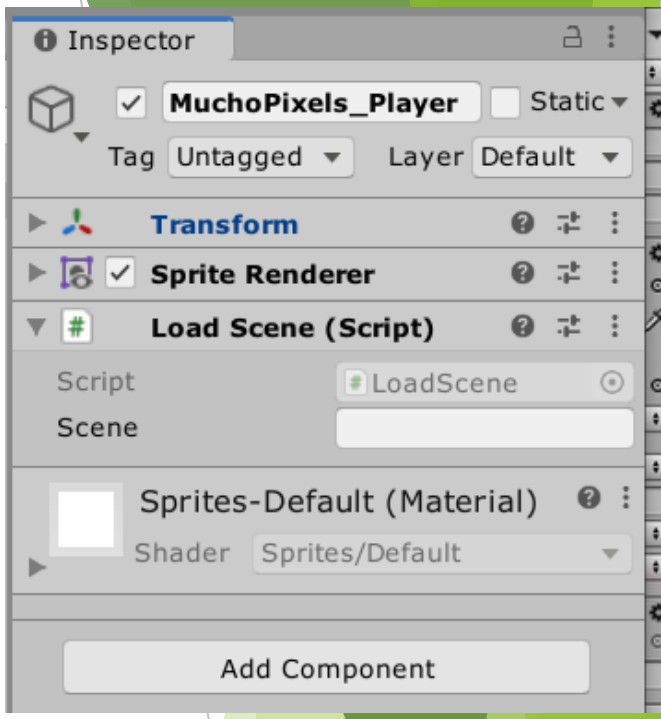
- Vamos criar um novo Asset, o Script LoadScene.
- Clique com o botão direito na pasta Scripts da janela Project. Em seguida, clique em Create > C# Script.
- Digite o nome do objeto criado como LoadScene. Isso criará o arquivo LoadScene.cs, cujo conteúdo digitaremos a seguir.
- Clique duas vezes sobre o objeto LoadScene, para abrir o editor de textos associado à sua instalação do Unity (poderá ser o Visual Studio Community, o Visual Studio Code ou outro que você tenha configurado com o menu Edit | Preferences | External Script Editor.
- Apague os tratadores de evento Start() e Update(), que estão no código, e digite o código interno da classe LoadScene como vemos ao lado.
- Temos uma variável pública de tipo string, chamada scene e um tratador do evento OnMouseDown(), que utiliza o método LoadScene do objeto SceneManager para carregar a cena cujo nome será armazenado na variável scene.
- É importante seguir a disposição de letras minúsculas e maiúsculas nessa digitação. Salve o arquivo e volte ao Unity.

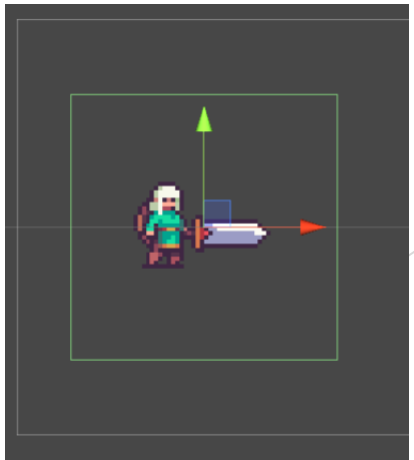


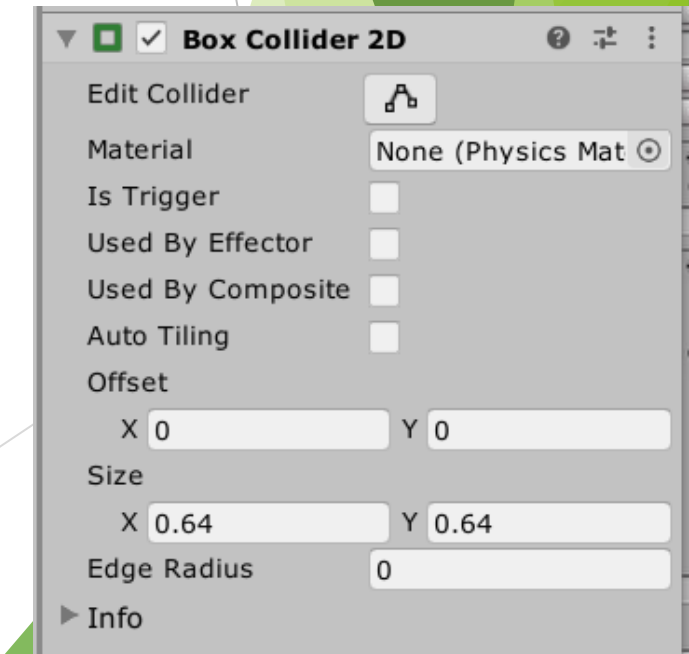


Prática 1

- Agora associaremos esse script com o personagem.
- Para isso, selecione o personagem na janela Scene, de forma que o Inspector tenha as propriedades desse objeto.
- Em seguida, arraste o script, desde a janela Project até a janela Inspector (que já está associada ao personagem).
- Assim, o personagem terá dentro de suas propriedades esse script, e será o objeto que controla esse Script.
- Como o script LoadScene faz um tratamento do evento OnMouseDown(), quando se clicar sobre o personagem (que controla o script), esse evento será detectado e o método que o trata será executado.
- Para que nosso personagem possa detectar que colidiu com algum outro objeto em cena e, também, detectar eventos de mouse, precisamos acoplar a ele um novo componente: um Box Collider 2D.
- No Inspector, clique em [Add Component], selecione a opção Physics 2D e Box Collider 2D.
- Aparecerá um quadrado verde ao redor do personagem. Ele indica o detector de colisões.
- Salve a cena alterada, usando Ctrl-S ou File | Save.

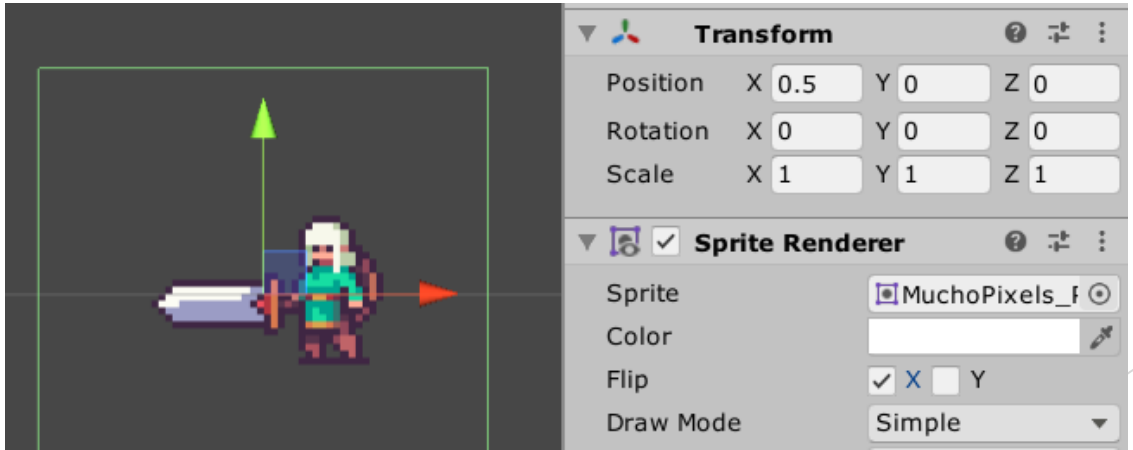


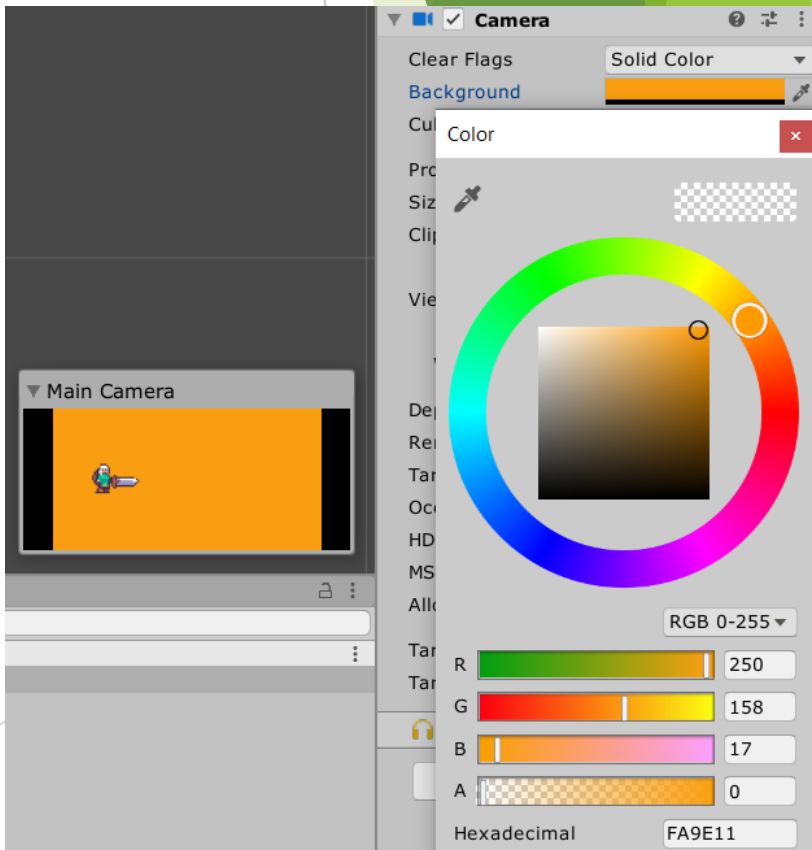


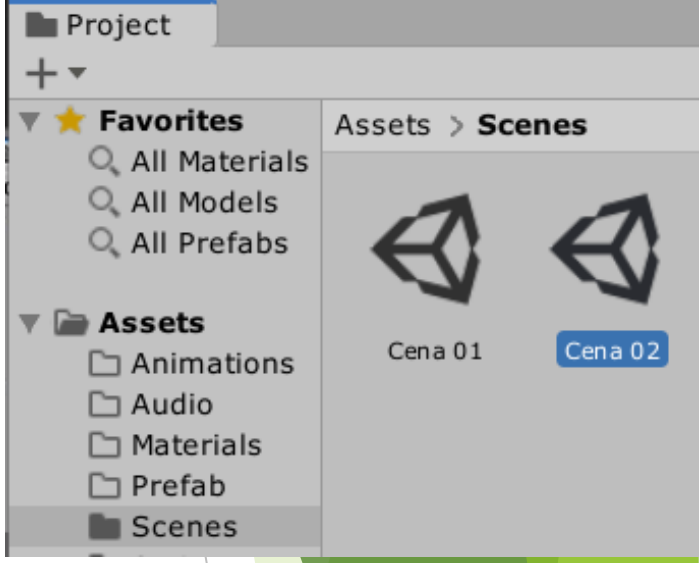


Prática 1

- Agora que salvamos a cena 01, com suas alterações, vamos duplica-la para termos uma segunda cena, que será carregada pelo evento MouseDown() do personagem.
- Para isso, abra a pasta Scenes da janela Project , clique no objeto Cena 01 e selecione a opção do menu Edit | Duplicate ou digite [Ctrl-D] com o teclado.
- Teremos uma cópia da cena. Se necessário, mude seu nome para Cena 02.
- Clicando duas vezes nesse objeto (Cena 02), abrirá a cena e a barra superior do Unity mostrará que essa é a cena aberta atualmente.
- Para diferenciar um pouco uma cena da outra, vamos mudar a Camera Principal. Mude a propriedade BackGround da Main Camera para uma cor mais bonita.
- Selecione o personagem, mude Position.X para 0.5 e ligue Flip.X no Sprite Renderer:
- Salve a cena.

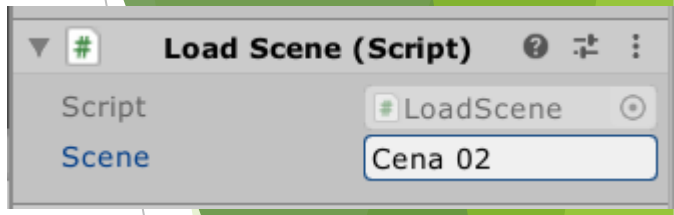


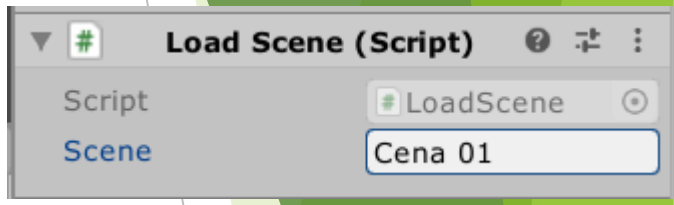


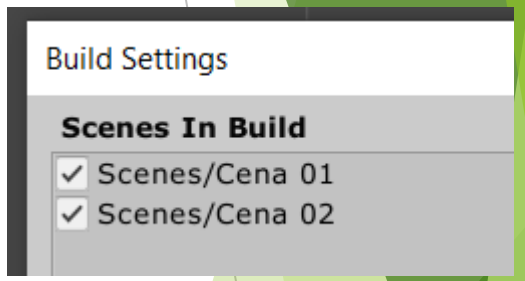


Prática 1

- No personagem de cada uma das cenas, no script LoadScene, indique o nome da outra cena na variável Scene, como vemos nas duas figuras ao lado.
- A primeira figura é da Cena 01 e a segunda figura é da Cena 02.
- Salve cada cena após fazer essa configuração.
- Abra a Cena 01 e execute o jogo.
- Clique no personagem e não funcionará. Uma mensagem vermelha aparecerá na parte inferior da janela do Unity. Ela informa que a cena 02 não foi lista nas cenas a serem compiladas. Clique nela para ver a janela Console.
- Vá no Build Settings e arraste a Cena 02 para a lista Scenes in Build.
- Reabra a Cena 01 e execute o jogo.
- Clique no personagem e observe que a cena 02 é carregada.
- Clique no personagem dessa cena e observe que a cena 01 é recarregada.

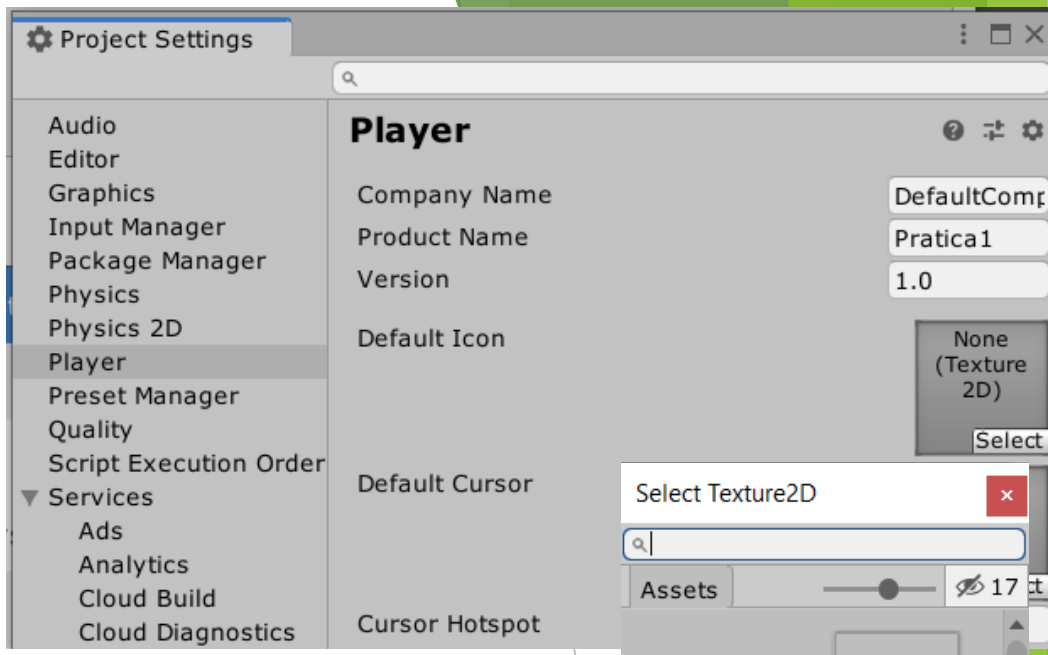


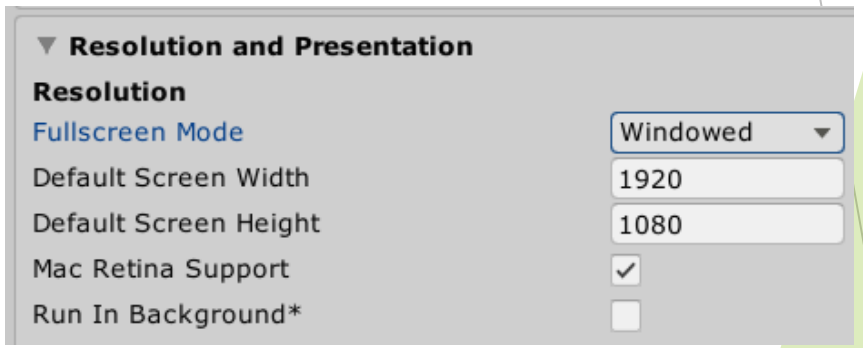


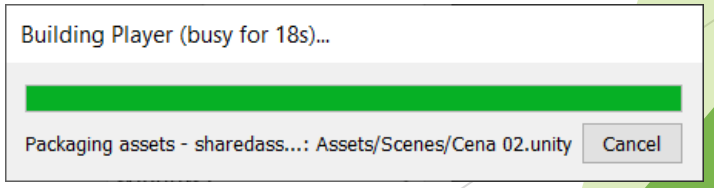


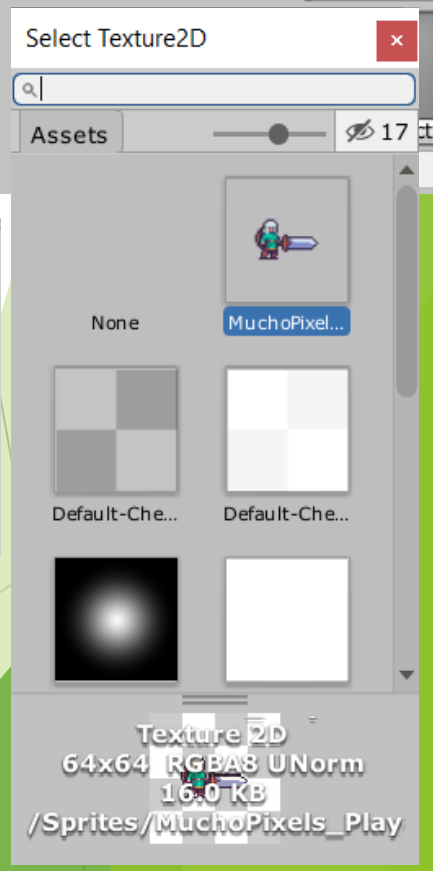
Prática 1

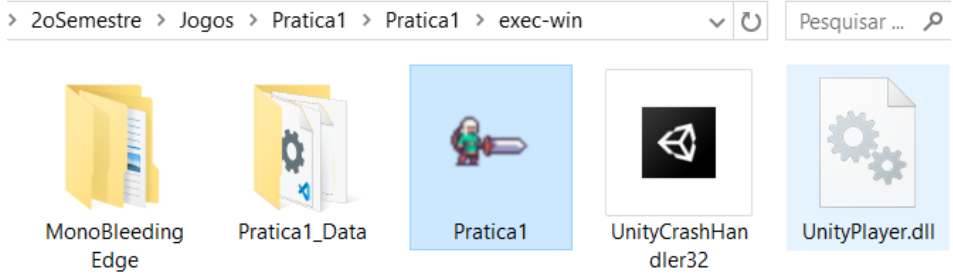
- Vamos criar um ícone para nosso jogo e gerar o executável.
- Acesse a opção de Menu File | Build Settings | Player Settings.
- Vá à janela Projects, acesse a pasta Sprites e arraste o sprite do nosso personagem para o quadro Default Icon da janela Player Settings. Alternativamente, clique no botão [Select] desse quadro e selecione o arquivo desejado.
- Mude também o item Resolution and Presentation, indicando Fullscreen Mode como Windowed.
- Feche a janela Player Settings e clique no botão [Build] da janela Build Settings. Ela solicitará a pasta onde o executável do jogo será gerado. Indique uma pasta e aguarde a compilação (leva um certo tempo).
- Ao final dela, a pasta do executável será apresentada. Note como o ícone escolhido aparecerá. Clique no aplicativo gerado e o execute.





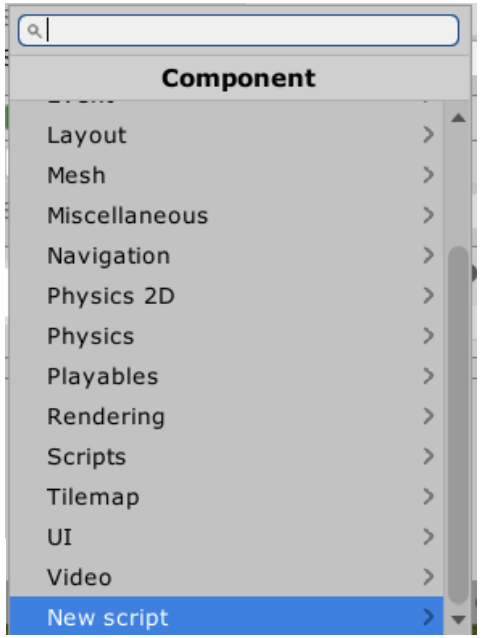






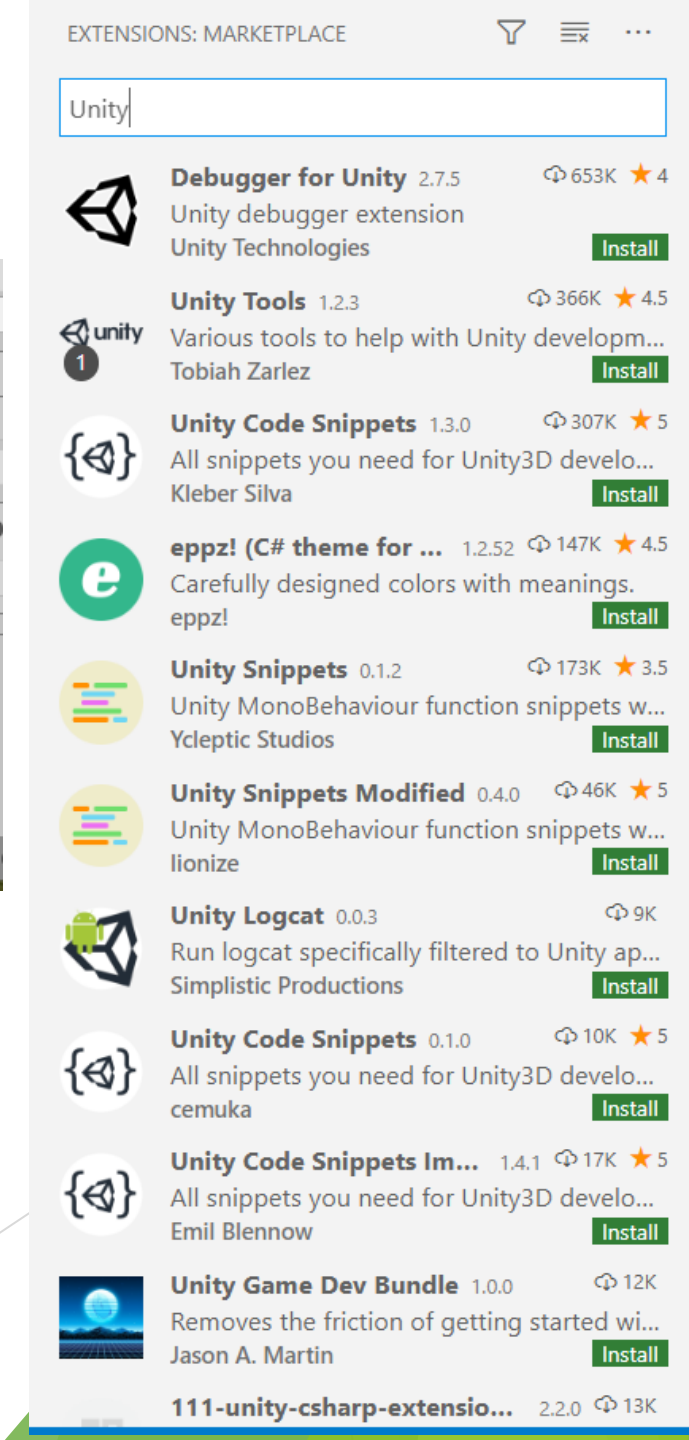
Prática 1

- Se você estiver usando o Visual Studio Code, pode instalar algumas das extensões da figura ao lado.
- Vamos incrementar um pouco nosso personagem. Na Cena 01, clique no personagem e, no Inspector, clique no botão [Add Component] e escolha New Script.
- Digite o nome do script como FazAndar. Aparecerá um novo item no Inspector do personagem: o script FazAndar.cs.
- Clique no nome dele para abrir o editor de código. No tra-tador de evento Update, digite o código abaixo e salve o arquivo:




```
void Update()
{
    float x = Input.GetAxis("Horizontal");
    float y= Input.GetAxis("Vertical");
    Vector2 movimento = new Vector2(x, y) * Time.deltaTime;
    transform.Translate(movimento);
}
```

- Esses comandos capturam a interação do jogador com o jogo através do controla-dor e das teclas (W, A, S, D, setas, espaço). Cria um vetor de deslocamen-to 2D e faz o objeto proprietário do script (transform) transladar por esse vetor. Execute.
- Vídeo tutorial sobre GetAxis()



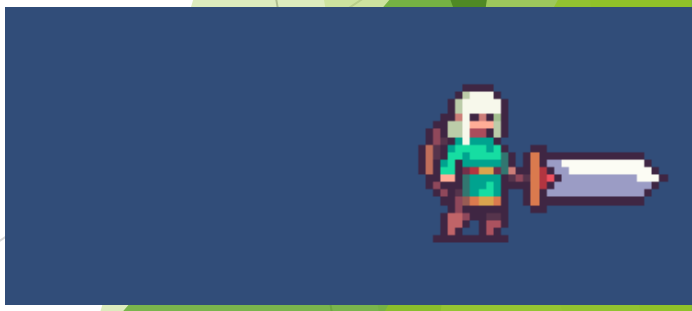
Prática 1

- Você deve ter conseguido fazer seu personagem andar para cima, para baixo, para a direita e para a esquerda, usando as setas ou o controlador de jogo.
- Vamos melhorar um pouco o jogo. No mesmo tratador de evento Update() que codificamos antes, digite o comando abaixo, logo após obter o valor de x:

```
void Update()
{
    float x = Input.GetAxis("Horizontal");
    GetComponent<SpriteRenderer>().flipX = x < 0;
    float y= Input.GetAxis("Vertical");
```

GetComponent é um método de um game object que busca o componente entre < e > e permite acessar e alterar suas propriedades. Neste acaso, pertence ao objeto dono do script. flipX é boolean.

- Salve o script e execute o jogo. Observe que, ao usar a seta para a esquerda ou a tecla A, o personagem se move para a esquerda mas, também, gira ao redor do eixo Y, passando a apontar para a direção em que anda.
- Quando você usa a tecla D ou a seta para a direita, o personagem se move nessa direção e sua frente também aponta nessa direção.



- Mas, quando paramos o movimento, o personagem sempre apontará para a direita, mesmo tendo sido movido para a esquerda. Isso ocorre porque, quando não mexemos com o personagem, Input.GetAxis("Horizontal") retorna 0, e 0 não é menor que 0, e a expressão lógica resulta em false, removendo o flip.
- Vamos resolver essa situação no próximo slide.

Prática 1

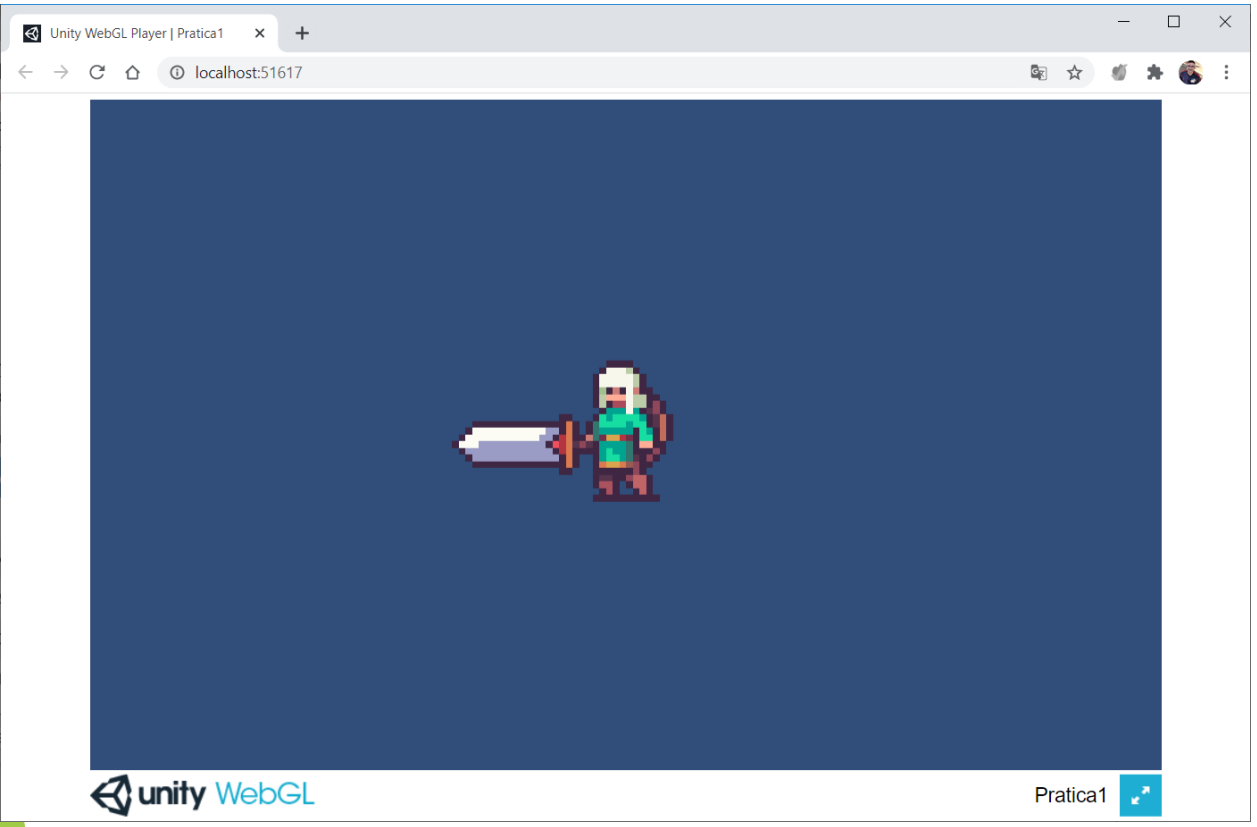
- Declare a variável personagemSpriteRenderer, cujo tipo é a classe SpriteRenderer. Ela será iniciada no evento Awake(). Esse evento ocorre quando o objeto ativo que contém o script é inicializado no momento em que a cena é carregada. Quando um objeto previamente inativo é reativado, Awake() é disparado.
- PersonagemSpriteRenderer, portanto, referencia o componente SpriteRenderer do personagem, que é responsável pela geração de sua imagem no jogo.
- No evento Update(), mudamos a atribuição de flipX do Sprite Renderer para verdadeiro ou falso, de acordo com o lado para o qual o personagem se move. Se for 0, sua frente apontará para a última direção na qual se moveu.
- Unity procura executar o evento Update() 60 vezes por segundo, para que seja possível ao olho do jogador capturar um movimento fluido.
- Como cada GPU pode ter uma velocidade diferente e é muito mais rápida que o olho humano, usamos Time.deltaTime para homogeneizar a velocidade de movimentação independentemente da GPU.

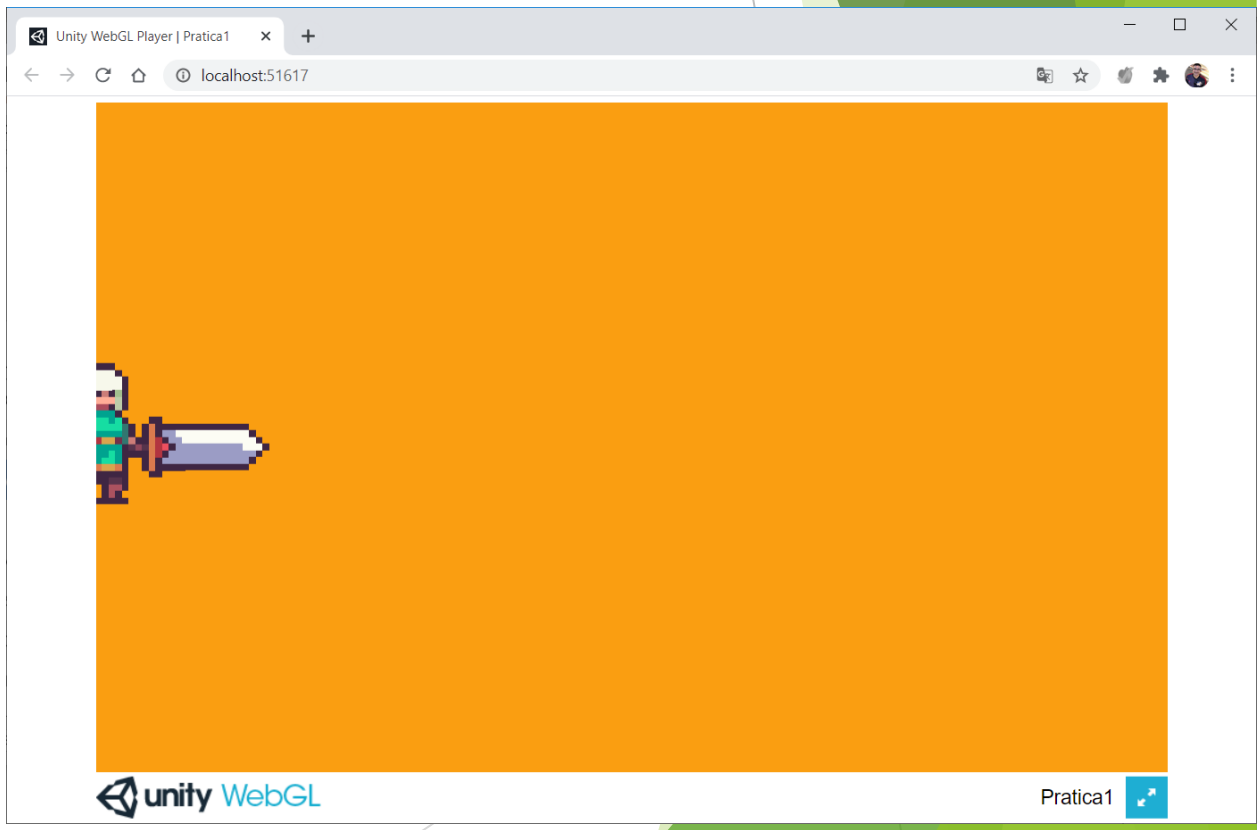
```
public class FazAndar : MonoBehaviour
{
    // Start is called before the first frame update
    3 references
    private SpriteRenderer personagemSpriteRenderer;
    0 references
    void Awake()
    {
        personagemSpriteRenderer = GetComponent<SpriteRenderer>()
    }

    // Update is called once per frame
    0 references
    void Update()
    {
        float x = Input.GetAxis("Horizontal");

        if (x < 0)  // vira para a esquerda
           personagemSpriteRenderer.flipX = true;
        else
          if (x > 0)  // vira para a direita
             personagemSpriteRenderer.flipX = false;
             // se for 0, fica como está

        float y= Input.GetAxis("Vertical");
        Vector2 movimento = new Vector2(x, y) * Time.deltaTime;
        transform.Translate(movimento);
    }
}
```

Prática 1

- Salve essa cena, entre na Cena 02 e faça com que o personagem dessa cena tenha o mesmo comportamento que o da Cena 01.
- Para isso, basta associar o mesmo script (FazAndar.cs) a esse personagem.
- Não é preciso criar outro script.
- Essa associação pode ser feita arrastando-se o script desde a pasta Scripts da janela Projects até o próprio personagem ou até o Inspector (desde que o personagem tenha sido selecionado previamente na janela Scene).
- Pudemos usar o mesmo script sem ter que alterá-lo, para identificar o personagem exato que estava sendo afetado pelo script, porque o uso de GetComponent, por exemplo, se refere ao objeto de jogo que está associado ao script. Da mesma forma, o objeto **transform** que usamos no código (sem ter de declará-lo) se refere, também, a essa propriedade do objeto associado ao script.
- Como, de uma cena para a outra, apenas mudamos o objeto associado ao script, esse objeto reage ao script de forma igual ao que o outro objeto fazia na cena anterior.
- Salve a Cena 02, compile para WebGL com [Build and Run] e execute o seu jogo através do browser.

Prática 1


Unity WebGL Player | Pratica1
localhost:51617
unity WebGL
Pratica1


Unity WebGL Player | Pratica1
localhost:51617
unity WebGL
Pratica1